Gestión de Soluciones - I

0.1. Introducción: de los datos al conocimiento

El **dato** es una representación sintáctica, generalmente numérica, que puede manejar un dispositivo electrónico - normalmente un ordenador - sin significado por sí solo. Sin embargo, el dato es a su vez el ingrediente fundamental y el elemento de entrada necesario en cualquier sistema y/o proceso que pretenda extraer información o conocimiento sobre un dominio determinado. En este sentido, 7 es un dato, como también lo es π o como son los términos *aprobado* o *suspenso*.

Por su parte, la **información** es el dato interpretado, es decir, el dato con significado. Para obtener información, ha sido necesario un proceso en el que, a partir de un dato como elemento de entrada, se realice una **interpretación** de ese dato que permita obtener su significado, es decir, información a partir de él. La información es también el elemento de entrada y de salida en cualquier proceso de toma de decisiones. Partiendo de los datos del ejemplo anterior, información obtenida a partir de los mismos puede ser: *El 7 es un número primo*, *π es una constante cuyo valor es* $3,141592653...$, *María ha aprobado el examen de conducir*, *Pablo está suspenso en matemáticas*.

A partir de información, es posible construir **conocimiento**. El conocimiento es información aprendida, que se traduce a su vez en reglas, asociaciones, algoritmos, etc. que permiten resolver el proceso de toma de decisiones. Así pues, la información obtenida a partir de los datos permite generar conocimiento, es decir, aprender. El conocimiento no es estático, como tampoco lo es siempre el aprendizaje. Aprender, construir conocimiento, implica necesariamente contrastar y validar el conocimiento construido con nueva información que permita, a su vez, guiar el aprendizaje y construir conocimiento nuevo. Siguiendo con los ejemplos anteriores, el conocimiento que permite obtener que el 7 es un número primo puede ser

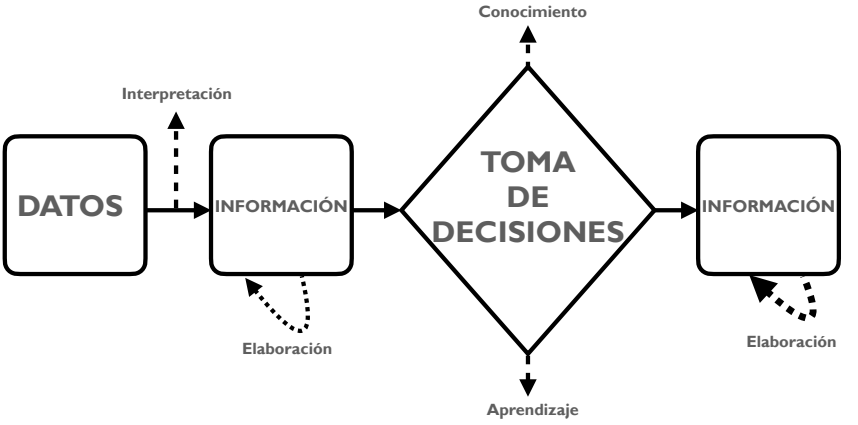


Figura 1: Relación entre datos, información y conocimiento en el proceso de toma de decisiones.

el *algoritmo de Eratóstenes*. Por otra parte, el conocimiento que permite obtener el valor del número π puede extraerse de los resultados de los trabajos de *Jones*, *Euler* o *Arquímedes*, mientras que el aprobado de María en el examen de conducir y el suspenso de Pablo en matemáticas, se pueden obtener de la regla que en una escala de diez asigna el aprobado a notas mayores o iguales que 5 y el suspenso a notas menores.

Por tanto, datos, información y conocimiento están estrechamente relacionados entre sí y dirigen cualquier proceso de **toma de decisiones** (ver [AN95], [FPSS96]). La figura 1 muestra la relación entre datos, información y conocimiento, en un proceso genérico de toma de decisiones. Más concretamente, en el ejemplo del suspenso de Pablo en matemáticas, el proceso de toma de decisión acerca de la calificación de Pablo se estructuraría de la siguiente forma:

1. El profesor corrige el examen de Pablo, que ha sacado un 3. Esta calificación, por sí sola, es simplemente un dato.

2. A continuación, el profesor calcula la calificación final de Pablo, en base a la nota del examen, sus trabajos y prácticas de laboratorio. *La nota final de Pablo es un* 4. Esto último es información.

3. ¿Ha aprobado Pablo? La información de entrada al proceso de decisión es su calificación final de 4 puntos, obtenida en el paso anterior. El conocimiento del profesor sobre el sistema de calificación le indica que una nota menor a 5 puntos se corresponde con un suspenso y, en caso contrario, con un aprobado.

4. La información de salida tras este proceso de decisión es que *Pablo está suspenso en matemáticas*.



Pregunta. Siguiendo el ejemplo anterior ¿Cómo se produce el proceso de toma de decisiones para determinar si un número es primo?

Aunque se trate de un ejemplo trivial, la importancia del proceso de toma de decisiones no lo es. En **marketing**, por ejemplo, se analizan bases de datos de clientes para identificar distintos grupos e intentar predecir el comportamiento de estos. En el mundo de las **finanzas**, las inversiones realizadas por grandes empresas responden a un proceso complejo de toma de decisiones donde los datos son el eje fundamental de este proceso. En **medicina**, existe una gran cantidad de sistemas de ayuda a la decisión que permiten a los doctores contrastar y validar sus diagnósticos de forma precoz. En definitiva, no hay área de conocimiento ni ámbito de aplicación que escape al proceso de toma de decisiones.

0.2. La carrera entre los datos y la tecnología

Que los datos son el elemento fundamental en cualquier proceso y/o sistema de **toma de decisiones** no es algo nuevo. Sin embargo, los datos no siempre han estado al alcance de los expertos y no siempre ha sido posible ni sencillo procesarlos según las necesidades concretas de cada caso de aplicación.

La información, por tanto, siempre ha sido poder y el gran reto ha sido y sigue siendo extraer información a partir de datos para generar conocimiento. Para ello, es necesario contar con dos factores que deben estar alineados: **datos y tecnología**.

Obtener **datos** no ha sido siempre una tarea fácil. Esto es debido principalmente a que la gran cantidad de sensores disponibles en la actualidad, que permiten registrar magnitudes de cualquier proceso, no existía como a día de hoy. Además, los sensores existentes en esta época (finales del siglo XX y comienzos del siglo XXI) no estaban ampliamente extendidos, ya que sus prestaciones estaban lejos de las que ofrecen hoy y sus precios no estaban al alcance de cualquier usuario. Por tanto, los procesos que se monitorizaban y de los cuales se recogían datos eran, sobretodo, procesos industriales realizados en grandes empresas. Por todos estos motivos, tradicionalmente se recurría a **modelos de simulación** que, a través de la implementación de un modelo matemático, permitían generar datos realistas de un proceso. Así, por ejemplo, se han generado datos de consumo eléctrico, tráfico urbano... Los datos generados mediante simulación son conocidos como **datos sintéticos** mientras que los datos provenientes de las lecturas de un sensor se conocen como **datos reales**.

Pero con los datos no es suficiente. Es necesario también contar con la **tecnología** necesaria para su procesamiento. Generar, almacenar y procesar todos estos datos no es una tarea trivial. Así pues, el **almacenamiento** se presenta como el primer problema tecnológico a resolver. Algunas soluciones propuestas pasan por los

sistemas de información distribuida, entendidos como un conjunto de ordenadores separados físicamente y conectados en red destinados al almacenamiento de datos o por los **sistemas de información en la nube**, que permiten adquirir espacio de almacenamiento en servidores privados, dejando la gestión de estos servidores en manos del proveedor. El segundo problema tecnológico es el **procesamiento** de los datos almacenados. Este aspecto cobra especial relevancia en función del caso de aplicación, pudiendo distinguirse entre procesamiento *on-line* (en línea) y procesamiento *off-line* (fuera de línea). En el caso del procesamiento ***on-line***, los datos son procesados a medida que son generados, ya que se requiere una respuesta en tiempo real. Por ejemplo, en un sistema de control del tráfico que permite regular los semáforos en función del tráfico actual, el sistema debe regular el semáforo a medida que se van generando e interpretando los datos del tráfico en un instante de tiempo dado. Por otra parte, en el caso del procesamiento ***off-line***, no es necesario que los datos se procesen a medida que se generan. Por ejemplo, en un sistema de detección del fraude bancario, comprobar si un cliente ha realizado algún movimiento fraudulento es una tarea que puede llevarse a cabo *off-line*, por ejemplo, haciendo un análisis de los movimientos del cliente en un momento dado, sin tener por qué diagnosticar cada movimiento que este va realizando. En este sentido, la **computación distribuida**, en donde múltiples máquinas realizan el procesamiento optimizando el rendimiento o la **computación en la nube**, que permite adquirir recursos de procesamiento al igual que se puede adquirir espacio de almacenamiento, son dos soluciones al problema del procesamiento. Otras alternativas son la **programación paralela** y la **programación multi-procesador**, que permiten, respectivamente, aprovechar el paralelismo de múltiples hilos de ejecución dentro de un procesador y realizar el procesamiento dividiéndolo en múltiples hilos en diferentes procesadores (ver [MSC13]).



Pregunta. Piensa en procesos cotidianos que requieran un procesamiento *on-line* y en otros que requieran un procesamiento *off-line*.

En la actualidad, la proliferación de una gran cantidad de sensores con altas prestaciones y precios asequibles que permiten monitorizar y generar datos sobre cualquier proceso ha supuesto un **incremento exponencial en la cantidad de datos generados**. En la actualidad, es posible monitorizar casi cualquier proceso, incluyendo los domésticos como el consumo eléctrico de un hogar, la presencia dentro del mismo o procesos cotidianos como la actividad física, entre otros muchos. Hoy, los datos llevan la delantera en la carrera entre datos y tecnología. Si bien es cierto que la tecnología ha experimentado grandes avances en los últimos años, la cantidad de datos generada no deja de crecer. Esto supone un **reto permanente para la tecnología**, que sigue evolucionando a nivel hardware con la aparición de arquitecturas con mayores posibilidades procesamiento y almacenamiento y a nivel software, con la aparición de modelos de programación que optimizan el procesamiento de los datos.

0.3. Los datos: los de ayer y los de hoy

Al igual que la tecnología ha ido evolucionando para dar respuesta a la ingente cantidad de datos que ha comenzado a generarse, estos últimos también han experimentado una gran evolución. Esta evolución, o revolución, no está únicamente relacionada con la **cantidad** de datos (como se expuso en el anterior apartado) sino también con el **tipo** y el **formato** de los mismos.

Tradicionalmente, el tipo y formato de datos con el que se ha trabajado para extraer información y conocimiento a partir de ellos era ciertamente **limitado**. En muchas ocasiones se trataba de ficheros de datos estructurados de forma tabular, donde cada fila del conjunto de datos representaba una instancia del mismo y cada columna una variable o atributo de la instancia. El formato de archivo que se manejaba solían ser **formatos de hojas de cálculo** (.xlsx, .ods, .numbers etc) o **ficheros separados por comas** (.csv). Muy pocos eran los procesos en los que se trabajaba con otros tipos de datos como texto, imágenes, audio e incluso vídeos, ya que los formatos de estos tipos de datos eran limitados hace unos años, su procesamiento más complejo y la tecnología para ello aún en desarrollo.

Aunque a día de hoy también se sigue trabajando con archivos de datos en forma de hojas de cálculo y/o archivos tradicionales para generar conocimiento a partir de ellos, las posibilidades actuales son prácticamente **ilimitadas**. En cuanto al texto, las técnicas de inteligencia artificial y procesamiento del lenguaje natural hacen posible la extracción de conocimiento a partir de **grandes volúmenes de textos**, que pueden provenir de páginas web, archivos .pdf, redes sociales, etc. El desarrollo de hardware con mejores prestaciones y los nuevos modelos de programación permiten procesar en la actualidad **grandes cantidades de imágenes, audios y vídeos** con una gran variedad de técnicas de inteligencia artificial en tiempos razonables. Finalmente, han aparecido nuevos tipos y formatos de datos, como por ejemplo, aquellos datos generados a partir de **grafos**, los cuales se tratarán en próximas secciones y capítulos con más detenimiento. Estos datos se corresponden, por ejemplo, con datos geográficos obtenidos a partir de mapas como los generados en aplicaciones como *Google Maps* u *Open Street Maps* o datos de seguimiento y actividad en redes sociales de gran valor en campañas publicitarias entre otros muchos (ver [HT14]).



Pregunta. Haz una búsqueda y elabora un listado con distintos tipos de datos y los formatos de almacenamiento más utilizados con los que se trabaja en ciencia de datos y big data.

Los diferentes tipos y formatos de datos, los de ayer y los de hoy, son la materia básica fundamental en cualquier proceso de extracción de información y de conocimiento. Después, las metodologías empleadas para ello y arquitecturas hardware sobre las que se realice el procesamiento de los mismos, permitirán definir **procesos y metodologías de big data**, aplicadas a un ámbito concreto.

0.4. Soluciones Big Data

En esta nueva era tecnológica en la que nos hayamos inmersos, a diario se generan enormes cantidades de datos, del orden de **petabytes** (más de un millón de gigabytes). Hoy en día, cualquier dispositivo como puede ser un reloj, un coche, un smartphone, etc está conectado a Internet generando, enviando y recibiendo una gran cantidad de datos. Tanto es así, que se estima que el $90\,\%$ de los datos disponibles en el mundo ha sido generado en los últimos años. Sin lugar a dudas, esta y las próximas generaciones serán las generaciones del **big data** (ver [HT14]).

Esta realidad descrita anteriormente demanda la capacidad de enviar y recibir datos e información a gran velocidad, así como la capacidad de almacenar tal cantidad de datos y procesarlos en tiempo real. Así pues, la gran cantidad de datos disponibles junto con las herramientas, tanto hardware como software, que existen a disposición para analizarlos se conoce como **big data**.

No existe una definición precisa del término **big data**, ni tampoco un término en castellano que permita denominar este concepto. A veces se usan en castellano los término *datos masivos* o *grandes volúmenes de datos* para hacer referencia al big data. Por este motivo, a menudo el concepto de big data es definido en función de las características que poseen los datos y los procesos que forman parte de este nuevo paradigma de computación. Esto es lo que se conoce como las Vs del big data (ver [BNCD16]).

Algunos autores coinciden en que big data son datos cuyo **volumen** es demasiado grande como para procesarlos con las tecnologías y técnicas tradicionales, requiriendo nuevas arquitecturas hardware, modelos de programación y algoritmos para su procesamiento. Además, se trata de datos que se presentan en una gran **variedad** de estructuras y formatos: datos sintéticos, provenientes de sensores, numéricos, textuales, imágenes, audio, vídeo... Finalmente, se trata de datos que requieren ser procesados a gran **velocidad** para poder extraer valor y conocimiento de ellos. Esta concepción se conoce como las tres Vs del big data (ver figura 2).

Otros autores amplían las características que han de tener los datos que forman parte del big data, incluyendo “otras Vs” como lo son: **volatilidad**, referida al tiempo durante el cual los datos recogidos son válidos y a durante cuánto tiempo deberán ser almacenados. **Valor**, referido a la utilidad de los datos obtenidos para

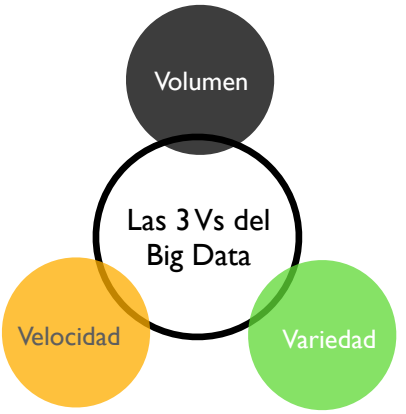


Figura 2: Definición de big data en base a “Las tres Vs del big data”.

extraer conocimiento y tomar decisiones a partir de ellos. **Validez**, referida a lo precisos que son los datos para el uso que se pretende darles. El uso de datos validados permitirá ahorrar tiempo en etapas como la limpieza y el preprocesamiento de los datos. **Veracidad**, relacionada con la confiabilidad del origen del cual provienen los datos con los que se trabajará así como la incertidumbre o el ruido que pudiera existir en ellos. **Variabilidad**, frente a la variedad de estructuras y formatos, hace referencia a la complejidad del conjunto de datos, es decir, al número de variables que contiene. Estas características, unidas a las tres Vs descritas anteriormente, se conocen como las ocho Vs del big data (ver figura 3).

Dado que no existe una definición uniforme para el término big data, muchos autores definen el término en función de aquellas características que consideran más relevantes, por lo que es común encontrar “las cinco Vs del big data”, “las siete Vs del big data” o “las diez Vs del big data” según cada autor, apareciendo distintos términos para describir el big data, como también pueden ser **visualización** o **vulnerabilidad**, entre otros.

Son muchas las **soluciones a nivel hardware y software**, que se han propuesto a los problemas derivados del almacenamiento y el procesamiento de big data. A continuación, se describen los fundamentos de tres de ellas, las cuales serán desarrolladas a nivel teórico, tecnológico y práctico en los siguientes capítulos.



Figura 3: Definición de big data en base a "Las ocho Vs del big data".

0.4.1. Almacenes de datos

Las **bases de datos relacionales** son colecciones de datos integrados, almacenados en un soporte secundario no volátil y con redundancia controlada. La definición de los datos y la estructura de la base de datos debe estar basada en un modelo de datos que permita captar las interrelaciones y restricciones existentes en el dominio que se pretende modelizar (ver [dMP99]). A su vez, un **Sistema Gestor de Bases de Datos (SGBD)** se compone de una colección de datos estructurados e interrelacionados (una base de datos) así como de un conjunto de programas para acceder a dichos datos (ver [SKS06]).

Las bases de datos tradicionales, siguiendo la definición anterior, están basadas generalmente en sistemas relacionales u objeto-relacionales. Para el acceso, procesamiento y recuperación de los datos, se sigue el modelo **Online Transaction Processing (OLTP)**. Una transacción es una interacción completa con un sistema de base de datos, que representa una unidad de trabajo. Así pues, una transacción representa cualquier cambio que se produzca en una base de datos. El modelo OLTP, traducido al castellano como **procesamiento de transacciones en línea**, permite gestionar los cambios de la base de datos mediante la inserción, actualización y eliminación de información de la misma a través de transacciones básicas que son procesadas en tiempos muy pequeños. Con respecto a la recuperación de información de la base de datos, se utilizan operadores clásicos (concatenación, proyección, selección, agrupamiento...) para realizar consultas básicas y sencillas (realizadas, mayoritariamente, en lenguaje SQL y extensiones del mismo). Finalmente,

las opciones de visualización de los datos recuperados son limitadas, mostrándose fundamentalmente los resultados de forma tabular y requiriendo un procesamiento adicional y más complejo en caso de querer presentar datos complejos.

La revolución en la generación, almacenamiento y procesamiento de los datos, así como la irrupción del **big data**, han puesto a prueba el modelo de funcionamiento, rendimiento y escalabilidad de las bases de datos relacionales tradicionales. En la actualidad, se requiere de soluciones integradas que aúnen datos y tecnología para almacenar y procesar grandes cantidades de datos con diferentes estructuras y formatos con el objetivo de facilitar la consulta, el análisis y la toma de decisiones sobre los mismos. En este sentido, la **inteligencia de negocio**, más conocida por el término inglés **business intelligence**, investiga en el diseño y desarrollo de este tipo de soluciones. La inteligencia de negocio puede definirse como la capacidad de una empresa de estudiar sus acciones y comportamientos pasados para entender dónde ha estado la empresa, determinar la situación actual y predecir o cambiar lo que sucederá en el futuro, utilizando las soluciones tecnológicas más apropiadas para optimizar el proceso de toma de decisiones (ver [IGG03]).

Estas nuevas soluciones requerirán un modelo de procesamiento diferente a **OLTP**. Esto es así, ya que el objetivo perseguido por la inteligencia de negocio está menos orientado al ámbito transaccional y más enfocado al ámbito analítico. Las nuevas soluciones utilizan el modelo **Online analytical processing (OLAP)**. La principal diferencia entre OLTP y OLAP estriba en que mientras que el primero es un sistema de procesamiento de transacciones en línea, el segundo es un sistema de **recuperación y análisis** de datos en línea. Por tanto, OLAP complementa a SQL aportando la capacidad de analizar datos desde distintas variables y dimensiones, mejorando el proceso de toma de decisiones. Para ello, OLAP permite realizar cálculos y consolidaciones entre datos de distintas dimensiones, creando modelos que no presentan limitaciones conceptuales ni físicas, presentando y visualizando la información de forma flexible, esto es, en diferentes formatos. Los sistemas OLAP están basados, generalmente, en sistemas o interfaces multidimensionales que proporcionan facilidades para la transformación de los datos, permitiendo obtener nuevos datos más combinados y agregados que los obtenidos mediante las consultas simples realizadas por OLTP. Al contrario que en OLTP, las unidades de trabajo de OLAP son más complejas que en OLTP y consumen más tiempo. Finalmente, en cuanto a la visualización de los mismos, los sistemas OLAP permiten la visualización y el análisis multidimensional a partir de diferentes vistas de los datos, presentando los resultados en forma matricial y con mayores posibilidades estéticas y visuales. La tabla 1 muestra un resumen con las principales diferencias entre los sistemas **OLTP** y **OLAP**.

Un **almacén de datos**, más conocido por el término **data warehouse** (en inglés), es una solución de **business intelligence** que combina tecnologías y componentes con el objetivo de ayudar al uso estratégico de los datos por parte de una organización. Esta solución debe proveer a la empresa, de forma integrada, de capa-

Tabla 1: Tabla resumen y comparativa entre OLTP y OLAP

	Bases de datos relacionales	**Soluciones Business Intelligence**
	OLTP	**OLAP**
Concepto	Sistema de procesamiento de transacciones en línea	Sistema de recuperación y análisis de datos en línea
Funciones	Gestión de transacciones: inserción, actualización, eliminación...	Análisis de datos para dar soporte a la toma de decisiones
Procesamiento	Transacciones cortas	Procesamientos de análisis complejos
Tiempo	Las transacciones requieren poco tiempo de ejecución	Los análisis requieren mayor tiempo de ejecución
Consultas	Simples, utilizando operadores básicos tradicionales	Complejas, permitiendo analizar los datos desde múltiples dimensiones
Visualización	Básica. Muestra los datos en forma tabular	Muestra los datos en forma matricial. Mayores posibilidades gráficas

cidad de almacenamiento de una gran cantidad de datos así como de herramientas de análisis de los mismos que, frente al procesamiento de transacciones, permita transformar los datos en información para ponerla a disposición de la organización y optimizar el proceso de toma de decisiones.

0.4.2. Bases de datos documentales u orientadas a documentos

La gran variedad y heterogeneidad en los tipos de datos almacenados y procesados en los últimos años ha puesto sobre la mesa la cuestión de si las bases de datos relacionales son el modelo más óptimo para trabajar con según qué tipos de datos. Como alternativa a ellas, en los últimos años han proliferado las bases de datos NoSQL.

NoSQL es el término utilizado para referirse a un tipo de bases de datos que permiten almacenar y gestionar tipos de datos que tradicionalmente han sido difíciles de gestionar por parte de las bases de datos relacionales. Así pues, NoSQL hace referencia a **bases de datos documentales, bases de datos orientadas a grafos, buscadores, etc**. En contraposición a las bases de datos relacionales, las NoSQL se caracterizan principalmente por:

- **Independencia del esquema:** Al contrario que en las bases de datos relacionales, no es necesario diseñar un esquema para definir los tipos y estructura de los datos almacenados, permitiendo acortar el tiempo de desarrollo y facilitando las modificaciones de la estructura interna de la base de datos.
- **No relacionales:** El concepto de relación de las bases de datos relacionales no existe en NoSQL. Por tanto, se trabaja con datos que no están normalizados, lo cual aporta flexibilidad en relación a los tipos y estructuras de datos que pueden ser almacenados.

- **Distribuidas:** La cantidad de datos almacenados requiere de su almacenamiento en múltiples servidores, ya que un único servidor por potente que sea no podrá procesar en tiempos razonables tal cantidad de información. Este hecho permite utilizar hardware sencillo, ya que al utilizar múltiples servidores no es necesario que todos ellos tengan grandes prestaciones.

Las **bases de datos documentales** trabajan con documentos, entendidos como una estructura jerárquica de datos que, a su vez, puede contener subestructuras. En una primera aproximación, es fácil pensar en que estos documentos pueden ser documentos de Microsoft Word, documentos pdf, páginas web... Las bases de datos documentales pueden, efectivamente, trabajar con estos tipos de documentos. Sin embargo, el término documento en este contexto posee un mayor nivel de abstracción.

Los **documentos** pueden consistir en datos binarios o texto plano. Es posible que se traten de datos semiestructurados, cuando aparecen en formatos como **JavaScript Object Notation (JSON)** o **Extensible Markup Language (XML)**. Por último, también pueden ser datos estructurados conforme a un modelo de datos particular como, por ejemplo, **XML Schema Definition (XSD)**.

Tabla 2: Tabla comparativa entre XML y JSON

	XML	**JSON**
Lenguaje fuente	SGML	JavaScript
Tipo Lenguaje	Orientado a datos	Orientado a documentos
Notación	Pesada	Ligera
Etiquetas inicio y fin	Sí	No
Comentarios	Sí	No
Espacios de nombres	Sí	No
Soporte tipos de datos	No	Sí

Actualmente, **XML** y **JSON** son los formatos de intercambio de datos más utilizados en el desarrollo de aplicaciones web. Sin embargo, existen importantes diferencias entre ellos: XML es una extensión del lenguaje **Standard Generalized Markup Language (SGML)** el cual es un lenguaje que permite la creación, organización y etiquetado de documentos. Por su parte, JSON es una extensión del lenguaje **JavaScript**. XML tiene una notación más pesada que JSON, siendo este último un lenguaje más ligero que admite tipos de datos y matrices. Por su parte, XML no proporciona tipos ni estructuras de datos, sino que contiene un conjunto de reglas que, mediante el uso de atributos y elementos, permite codificar un docu-

mento. Finalmente, JSON es un lenguaje orientado a datos mientras que XML es un lenguaje orientado a documentos. La tabla 2 muestra una comparativa de estos y otros aspectos de ambos lenguajes.

En **XML**, la estructura principal de un documento está formada por dos elementos: el **prólogo** (opcional) y el **cuerpo**. El prólogo contiene a su vez dos partes: la declaración XML que establece la versión del lenguaje, el tipo de codificación y si se trata de un documento autónomo y la declaración del tipo de documento. El cuerpo, por su parte, contiene la información del documento (ver [TR03]).

Supongamos que Carlos ha enviado un mensaje de whatsapp a Javier diciéndole que han quedado con los compañeros de trabajo a las diez de la noche en la puerta del Sol. Un **documento XML** que representa este mensaje como un documento, podría ser el mostrado en el listado 1.

Listado 1: Quedada en la puerta del sol

```
1 <?xml version="1.0" encoding="ISO-8859-1"?>
2 <whatsapp>
3     <para> Javier </para>
4     <de>   Carlos </de>
5     <titulo> Quedada </titulo>
6     <contenido> A las 22:00 pm en la puerta del sol </contenido>
7 </whatsapp>
```

El listado 1 incluye en la primera línea el **prólogo** del documento, definiendo la versión y el tipo de codificación utilizada. A partir de la segunda línea, se define el **cuerpo** del documento que contiene, mediante etiquetas de apertura <>y cierre </>, los distintos atributos de los que se compone el documento.

En JSON, la sintaxis del lenguaje **tiene las mismas reglas que el lenguaje JavaScript**, del cual proviene. Sin embargo, los archivos JSON deben cumplir también otras reglas sintácticas adicionales. En primer lugar, un archivo JSON representará o bien un **objeto**, es decir, una tupla de pares clave-valor o bien una **colección de elementos**, es decir, un vector o array. Por otra parte, los archivos JSON que representan objetos comienzan con una llave de inicio { y terminan con una llave de cierre }. Cuando se representa un vector, sus elementos se encierran entre corchetes []. Las **cadenas** y **nombres de atributos** del objeto deberán encerrarse entre comillas, así como todos los nombres de los atributos del objeto, separándose cada elemento del siguiente con una coma (,) no habiendo una coma después del último elemento (ver [Fri19]).

Así pues, si se pretende representar en formato JSON el mensaje que ha enviado Carlos a Javier, el fichero JSON resultante sería el mostrado en el listado 2. Este fichero define un **objeto JSON** que contiene una serie de atributos entrecomillados cuyo valor asociado son cadenas de caracteres que, por tanto, también van entrecomilladas.

Listado 2: Quedada en la puerta del sol

```
{
   "para": "Javier",
   "de": "Carlos",
   "titulo": "Quedada",
   "contenido": "A las 22:00 pm en la puerta del sol"
}
```

Existen en la web múltiples aplicaciones online que permiten convertir archivos XML en JSON y viceversa. En sucesivos capítulos, se profundizará más sobre la sintaxis de estos dos lenguajes así como la equivalencia entre los mismos a la hora de definir documentos que serán tratados posteriormente por una base de datos documental. También se describirán las tecnologías más utilizadas en este ámbito, haciendo especial énfasis en **MongoDB**, uno de los sistemas de bases de datos NoSQL orientado a documentos más utilizado a día de hoy.

0.4.3. Bases de datos sobre grafos

Tal y como se expuso anteriormente, en los últimos años han aparecido nuevos tipos de datos como aquellos que vienen dados en forma de **grafo**. Algunos de estos datos son los provenientes de interacciones en **redes sociales, datos geográficos expresados en forma de mapas, etc.**

Un **grafo** es un ente matemático compuesto por un conjunto de **nodos o vértices** y un conjunto de **enlaces o aristas**. Matemáticamente puede ser expresado por medio de la ecuación (1).

$$G = \{V, E\} \tag{1}$$

Donde V representa el conjunto de nodos o vértices y E representa el conjunto de enlaces o aristas. Así pues, sea el grafo de la figura 4 que representa un conjunto de ciudades conectadas por autovías, el conjunto V de nodos sería $V = \{$La Coruña, Madrid, San Sebastián, Barcelona, Valencia, Sevilla, Cádiz$\}$, mientras que el conjunto de enlaces E vendría dado por $E = \{$A-1, A-2, A-3, A-4, A-4-I, A-4-II, A-6, A-7$\}$.

Una **base de datos orientada a grafos** es, por tanto, un sistema de bases de datos que implementa métodos de creación, lectura, actualización y eliminación de datos en un modelo expresado en forma de grafo. Existen dos aspectos fundamentales en este tipo de sistemas: el primero de ellos hace referencia al **almacenamiento de los datos**. En una base de datos orientada a grafos, los datos pueden almacenarse siguiendo el modelo relacional, lo que implica mapear la estructura del grafo a una

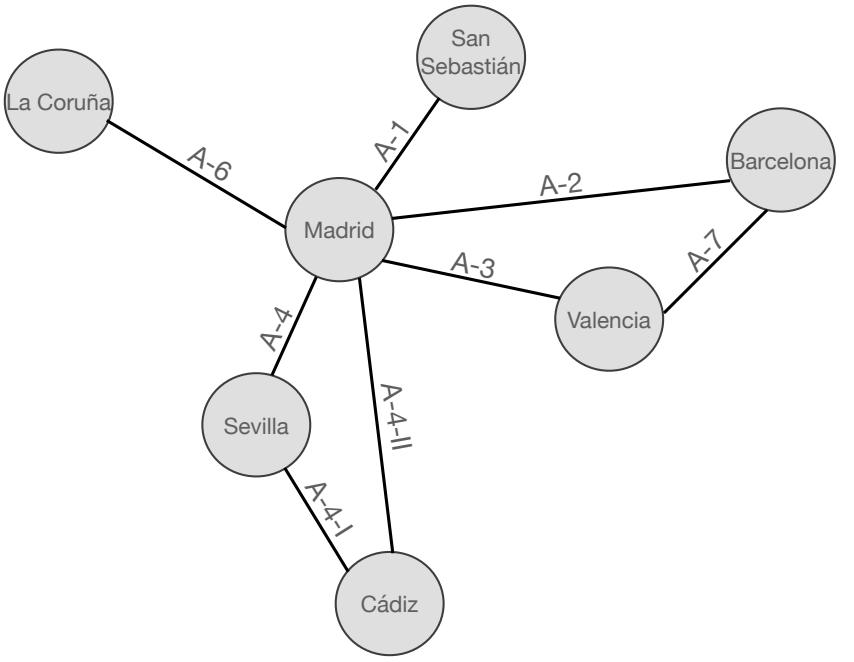


Figura 4: Grafo que representa las principales autovías de España

estructura relacional, o bien, almacenarse de forma **nativa** utilizando modelos de datos propios para almacenar estructuras de tipo grafo. La ventaja de mapear los grafos a una estructura relacional radica en que la gestión y consulta de los datos se realizará de forma tradicional a través de un backend conocido como, por ejemplo, MySQL. Por su parte, la ventaja del almacenamiento nativo de grafos radica es que existen modelos de datos e implementaciones que aseguran y garantizan el buen rendimiento y la escalabilidad del sistema. El segundo aspecto importante es el **procesamiento de los datos**. En este sentido, también es posible distinguir el procesamiento **no nativo** de los grafos, lo cual implica el tratamiento de estos datos siguiendo las técnicas tradicionales utilizadas en los lenguajes de modificación de datos de las bases de datos relacionales, o el procesamiento **nativo** de los datos de grafos, el cual es beneficioso porque optimiza los recorridos del grafo cuando se realizan consultas aunque, en ocasiones, invierta demasiado tiempo y memoria en consultas que no requieren de recorridos complejos (ver [RWE15]).

Si bien es cierto que **cualquier dominio puede ser modelado como un grafo**, esta no es una razón de peso como para cambiar o migrar de un esquema y modelo de datos bien conocido e investigado, como el esquema relacional, a un modelo orientado a grafos. Sin embargo, la motivación para requerir de sistemas de bases de datos específicos, orientados a trabajar con este tipo de datos, radica en tres aspectos principales:

- **Rendimiento:** Los sistemas de bases de datos orientados a grafos optimizan el rendimiento de las consultas sobre este tipo de datos con respecto a las bases de datos relacionales. Si bien es cierto que las consultas en el modelo relacional se vuelven más complejas y el rendimiento disminuye a medida que el conjunto de datos crece, esto no es así **cuando se trabaja con grafos, donde complejidad y rendimiento permanecen constantes**. Esto es así, ya que las consultas se localizan en una porción del grafo y, por tanto, solo es necesario explorar la parte del grafo afectada para resolver la consulta.

- **Flexibilidad:** Los sistemas de bases de datos orientados a grafos son más flexibles que los sistemas de bases de datos relacionales. Mientras que los primeros requieren la modelización exhaustiva a priori de todo el dominio, esto no es necesario en los segundos, donde **la naturaleza aditiva de los grafos** permite añadir nuevos nodos, relaciones, etiquetas y subgrafos a uno dado **sin necesidad de modificar todo el modelo de datos**, facilitando la implementación y reduciendo el riesgo a corromper el modelo de datos.

- **Agilidad:** A partir de las dos características anteriores, es posible deducir que el trabajo con sistemas de bases de datos orientados a grafos es más ágil que la gestión de estos datos a través de sistemas de bases de datos relacionales. Esta rápida gestión permitirá utilizar tecnología alineada con las nuevas metodologías de **desarrollo de software ágil**, permitiendo diseñar y desarrollar software que utilice este tipos de datos.

Existen distintos lenguajes de marcado de grafos, a partir de los cuales es posible generar archivos con formato de grafo para ser tratados por una base de datos de este tipo. Algunos de los lenguajes más utilizados son **GraphML, eXtensible Graph Markup and Modeling Language (XGMML), Graph Exchange Language (GXL) o Graph Modelling Language (GML)**, entre otros. La mayoría de ellos son variantes o extensiones del lenguaje XML para el modelado de grafos (ver [RB02]).

En la actualidad, **GraphML** (ver [BELP13]) es uno de los lenguajes más extendidos para el modelado de datos en forma de grafo. Se trata de un lenguaje sencillo, general, extensible y robusto que está compuesto por un núcleo del lenguaje que define las propiedades estructurales del grafo y un mecanismo flexible de extensiones que permite añadir información específica del mismo. La notación es muy similar a XML y se profundizará en ella en capítulos posteriores. A modo de ejemplo, el grafo mostrado en la figura 4 podría representarse en GraphML según se muestra en el listado 3.

Como se puede apreciar en el fragmento de código, en primer lugar se define un grafo cuyo identificador es Grafo_Autovías. El tipo de aristas que incluye este grafo son aristas **no dirigidas**, puesto que los enlaces del grafo no presentan flechas que indiquen dirección. Por tanto, los enlaces son bidireccionales. En caso de definir un grafo dirigido, el valor del atributo *edgedefault* sería *directed*. A continuación, se definen los distintos nodos que contendrá el grafo y después las aristas o enlaces,

definiendo el identificador de cada una de ellas así como su origen y destino. Al tratarse de un grafo no dirigido, origen y destino son intercambiables, no siendo así en caso de que se trate de un **grafo dirigido**.

Los sistemas de bases de datos orientados a grafos han proliferado mucho en los últimos años, existiendo numerosas tecnologías que dan soporte a ellos, como pueden ser **Microsoft Infinite Graph, Titan, OrientDB, FlockDB, AllegroGraph o Neo4j** que se estudiará con más profundidad posteriormente.

Listado 3: Grafo Autovías

```
1   <graph id="Grafo_Autovias" edgedefault="undirected">
2     <node id="La Coruña"/> <node id="San Sebastián"/>
3     <node id="Madrid"/>    <node id="Barcelona"/>
4     <node id="Valencia"/>  <node id="Sevilla"/>
5     <node id="Cadiz"/>
6     <edge id = "A-6"    source="La Coruña" target="Madrid"/>
7     <edge id = "A-1"    source="Madrid"    target="San Sebastián"/>
8     <edge id = "A-2"    source="Madrid"    target="Barcelona"/>
9     <edge id = "A-7"    source="Barcelona" target="Valencia"/>
10    <edge id = "A-3"    source="Madrid"    target="Valencia"/>
11    <edge id = "A-4"    source="Madrid"    target="Sevilla"/>
12    <edge id = "A-4-I"  source="Sevilla"   target="Cádiz"/>
13    <edge id = "A-4-II" source="Madrid"    target="Cádiz"/>
14  </graph>
```

Bibliografía

[AN95] Agnar Aamodt and Mads Nygård. Different roles and mutual dependencies of data, information, and knowledge — an ai perspective on their integration. *Data & Knowledge Engineering*, 16(3):191–222, 1995.

[BELP13] Ulrik Brandes, Markus Eiglsperger, Jürgen Lerner, and Christian Pich. Graph markup language (graphml). In Roberto Tamassia, editor, *Handbook on Graph Drawing and Visualization*, pages 517–541. Chapman and Hall/CRC, 2013.

[BNCD16] Rajkumar Buyya, Rodrigo N. Calheiros, and Amir Vahid Dastjerdi. *Big Data : Principles and Paradigms.* Elsevier Science & Technology, 50 Hampshire Street, 5th Floor, Cambridge. USA, 2016.

[dMP99] Adoración de Miguel and Mario Piattini. *Fundamentos y Modelos de Bases de Datos*. Alfaomega, 1999.

[FPSS96] Usama Fayyad, Gregory Piatetsky-Shapiro, and Padhraic Smyth. From data mining to knowledge discovery in databases. *AI magazine*, 17(3):37, 1996.

[Fri19] J. Friesen. *Java XML and JSON: Document Processing for Java SE*. Apress, 2019.

[GR09] Matteo Golfarelli and Stefano Rizzi. *Data Warehouse Design: Modern Principles and Methodologies*. Mcgraw-hill, 2009.

[HT14] Francisco Herrera Triguero. *Inteligencia Artificial, Inteligencia Computacional y Big Data*. Servicio de publicaciones. Universidad de Jaén, Jaén, 2014.

[IGG03] C. Imhoff, N. Galemmo, and J.G. Geiger. *Mastering Data Warehouse Design: Relational and Dimensional Techniques*. Wiley, 2003.

[Inm02] William H. Inmon. *Building the Data Warehouse,3rd Edition*. John Wiley & Sons, Inc., USA, 3rd edition, 2002.

[JVVL03] Matthias Jarke, Yannis Vassiliou, Panos Vassiliadis, and Maurizio Lenzerini. *Fundamentals of Data Warehouses*. Springer-Verlag, Berlin, Heidelberg, 1st edition, 2003.

[KR02] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit: The Complete Guide to Dimensional Modeling*. Wiley, 2 edition, April 2002.

[MSC13] Cukier Mayer Schönberger, Viktor and Kenneth Cukier. *Big data. La revolución de los datos masivos*. Turner Publicaciones, Madrid, 2013.

[MTK06] J. Mundy, W. Thornthwaite, and R. Kimball. *The Microsoft Data Warehouse Toolkit: With SQL Server 2005 and the Microsoft Business Intelligence Toolset*. Wiley, 2006.

[PVMCV06] Mario Piattini Velthuis, Esperanza Marcos, Coral Calero, and Belén Vela. *Tecnología y diseño de bases de datos*. Ra-Ma,, 2006.

[RB02] Pablo Rodríguez Bocca. Lenguajes canónicos para la descripción de grafos: Estudio y transformación entre esquemas de distintos modelos de datos. Interoperabilidad, Monografía, 2002.

[RWE15] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph Databases*. O'Reilly, 2 edition, 2015.

[SKS06] Abraham Silberschatz, Henry F. Korth, and Sudarshan S. *Fundamentos de bases de datos*. McGraw-Hill, Edificio Valrealty. 1 Planta. Basauri 17. 28023 Aravaca (Madrid), 2006.

[TR03] Erik T. Ray. *Learning XML*. O'Reilly, 2003.

Big Data Aplicado

Conforme a contenidos del «Curso de Especialización en Inteligencia Artificial y Big Data»



Universidad de Castilla-La Mancha

Escuela Superior de Informática
Ciudad Real

Capítulo 2

Gestión de Soluciones - II

Julio Alberto López Gómez

En este capítulo se profundizará en los **almacenes de datos** o *Data Warehouses* (DW) como solución propia de la **inteligencia de negocio** o *Business Intelligence* (BI). Esta solución aparece como respuesta al crecimiento de datos digitales almacenados por las empresas y la necesidad de extraer información y generar conocimiento a partir de ellos para **optimizar el proceso de toma de decisiones** en cada uno de los departamentos o divisiones de la empresa. Para ello, en este capítulo se aborda el **concepto de almacén de datos, su arquitectura así como su diseño e implementación.**

2.1. Sistemas de ayuda a la decisión

En una empresa u organización, los datos generados a diario son, principalmente, aquellos derivados de las operaciones rutinarias de la empresa. Estos datos, tradicionalmente, se almacenaban en **bases de datos relacionales** y su manipulación se correspondía con **transacciones** realizadas sobre la base de datos. Sin embargo, el objetivo de cualquier organización es seleccionar esos datos para realizar estudios y análisis que permitan generar informes que, a su vez, permitan a la empresa **extraer información para tomar decisiones estratégicas** que conduzcan a la organización al éxito.

El crecimiento exponencial de los datos manejados por una organización ha hecho que los computadores sean las únicas herramientas capaces de procesar estos datos para obtener información y ofrecer ayuda en la toma de decisiones. En este contexto, aparecen los **sistemas de ayuda a la decisión** o *Decision Support Systems* (DSS) que ayudan a quienes ocupan puestos de gestión a tomar decisiones o elegir entre diferentes alternativas (ver [GR09]).



Sistema de ayuda a la decisión: Conjunto de técnicas y herramientas tecnológicas desarrolladas para procesar y analizar datos para ofrecer soporte en la toma decisiones a quienes ocupan puestos de gestión o dirección en una organización. Para ello, el sistema combina los recursos de los gestores junto con los recursos computacionales para optimizar el proceso de toma de decisiones.

Mientras que las bases de datos relacionales han sido tradicionalmente el componente del *back-end* en el diseño de sistemas de ayuda a la decisión, los almacenes de datos se han convertido en una opción mucho más competitiva como elemento *back-end* al mejorar el rendimiento de éstas.

Los **campos de aplicación** de los almacenes de datos no se reducen únicamente al ámbito empresarial, sino que cubren multitud de dominios como las **ciencias naturales, demografía, epidemiología o educación, entre otros muchos.** La propiedad común a todos estos campos y que hace de los almacenes de datos una adecuada solución en estos ámbitos es **la necesidad de almacenamiento y herramientas de análisis que permitan obtener en tiempos razonables información y conocimiento útiles para mejorar el proceso de toma de decisiones** (ver [GR09]).

2.2. Almacenes de datos: Concepto

La aparición de los **almacenes de datos** está ligada, principalmente, a una serie de **retos** que es necesario abordar para **convertir los datos transaccionales** con los que trabaja una base de datos relacional en **información para generar conocimiento y dar soporte al proceso de toma de decisiones** (ver [GR09]).

- **Accesibilidad:** Desde cualquier dispositivo, a cualquier tipo de usuario y a gran cantidad de información que no puede ser almacenada de forma centralizada. La accesibilidad, en este sentido, debe hacer frente al problema de la **escalabilidad** del sistema y de los datos que este maneja.
- **Integración:** Referente a la gestión de datos **heterogéneos**, con distintos formatos, y provenientes de distintos ámbitos de la organización. Una correcta integración debe garantizar a su vez la **corrección y completitud** de los datos integrados.
- **Consultas mejoradas:** Permitiendo incluir operadores avanzados y dar soporte a herramientas y procedimientos que posibiliten obtener el máximo partido de los datos existentes. De este modo, será posible obtener **información precisa para realizar un análisis eficiente**.
- **Representación multidimensional:** Proporciona herramientas para analizar de forma multi-dimensional los datos del sistema, incluyendo datos de **diferentes unidades** de la organización con el objetivo de proporcionar herramientas de **análisis y visualización multi-dimensional** para mejorar el proceso de toma de decisiones.

A continuación, se muestra una **definición de almacén de datos** muy extendida, dada por W. Inmon, quien es conocido por ser el "padre" del concepto de almacén de datos.



Almacén de datos (Data Warehouse): Colección de datos orientados a temas, integrados, variante en el tiempo y no volátil que da soporte al proceso de toma de decisiones de la dirección (ver [Inm02]).

Para entender correctamente esta definición, es necesario ahondar en las características que incluye la misma (ver [PVMCV06]).

- **Orientados a temas:** Es decir, no orientado a procesos (transacciones), sino a entidades de **mayor nivel de abstracción** como "artículo" o "pedido".
- **Integrados:** Almacenados en un formato **uniforme y consistente**, lo que implica depurar o limpiar los datos para poder integrarlos.
- **Variante en el tiempo:** Asociados a un instante de tiempo (mes, trimestre, año...)
- **No volátiles:** Se trata de datos **persistentes** que no cambian una vez se incluyen en el almacén de datos.

El diseño y funcionamiento de los almacenes de datos se basa en el sistema de procesamiento analítico en-línea, **OLAP**. Este sistema se encarga del análisis, interpretación y toma de decisiones acerca del negocio, en contraposición a los sistemas de procesamiento de transacciones en línea, **OLTP**. Este sistema es encarnado por las bases de datos operacionales, las cuales ejecutan las transacciones de procesos operacionales del día a día.

Así pues, los sistemas **OLTP están dirigidos por la tecnología y orientados a automatizar las operaciones del día a día** de la organización, mientras que los sistemas **OLAP están dirigidos por el negocio y proporcionan herramientas para tomar decisiones a largo plazo**, mejorando la estrategia y la competitividad de la organización (ver [PVMCV06]). La tabla 2.1 muestra una comparativa entre las principales características de las bases de datos operacionales (OLTP) y los almacenes de datos (OLAP).

Tabla 2.1: Diferencias entre BBDD Operacionales y Almacenes de Datos (ver [GR09]).

Característica	BBDD Operacionales	Almacén Datos
Objetivo	Depende de la aplicación	Toma de decisiones
Usuarios	Miles	Cientos
Trabaja con...	Transacciones predefinidas	Consultas y análisis específicos
Acceso	Lectura y escritura a cientos de registros	Principalmente lecutra. Miles de registros
Datos	Detallados, numéricos y alfanuméricos	Agregados, principalmente numéricos
Integración	En función de la aplicación	Basados en temas, con mayor nivel de abstracción
Calidad	Medida en términos de integridad	Medida en términos de consistencia
Temporalidad Datos	Solo datos actuales	Datos actuales e históricos
Actualizaciones	Continuas	Periódicas
Modelo	Normalizado	Desnormalizado, multidimensional
Optimización	Para acceso OLTP a parte de la BBDD	Para acceso OLAP a gran parte de la BBDD

2.3. Almacenes de datos: Arquitectura

Las arquitecturas disponibles para el **diseño de almacenes de datos** se basan, principalmente, en garantizar que el sistema cumpla una serie de propiedades esenciales para su óptimo funcionamiento (ver [JVVL03] y [GR09]).

- **Separación:** De los datos transaccionales y los datos estratégicos que sirven como punto de partida a la toma de de decisiones.
- **Escalabilidad:** A nivel hardware y software, para actualizarse y garantizar el correcto funcionamiento del sistema a medida que el número de datos y usuarios aumenta.
- **Extensiones:** Permitiendo integrar e incluir nuevas aplicaciones sin necesidad de rediseñar el sistema completo.
- **Seguridad:** Monitorizando el acceso a los datos estratégicos guardados en el almacén de datos.



Las arquitecturas de almacenes de datos se clasifican, fundamentalmente, en dos tipos: arquitecturas **orientadas a la estructura** y arquitecturas **orientadas a la empresa**.

2.3.1. Arquitecturas orientadas a la estructura

Las arquitecturas orientadas a la estructura reciben su nombre debido a que están diseñadas poniendo especial énfasis en el **número de capas y elementos que componen la arquitectura del sistema de almacén de datos.** De acuerdo con este criterio, es posible distinguir las siguientes arquitecturas (ver [JVVL03],[MTK06], [GR09]).

Arquitectura de una capa

El objetivo principal de esta arquitectura, poco utilizada en la práctica, es **minimizar la cantidad de datos almacenados eliminando para ello los datos redundantes.** La figura 2.1 muestra un esquema de este tipo de arquitectura. En ella, **el almacén de datos creado es virtual**, existiendo un *middleware* que interpreta los datos operacionales y ofrece una vista multidimensional de ellos. **El principal inconveniente** de esta arquitectura es que su simplicidad hace que el sistema **no cumpla la propiedad de separación**, ya que los procesos de análisis se realizan sobre los datos operacionales.

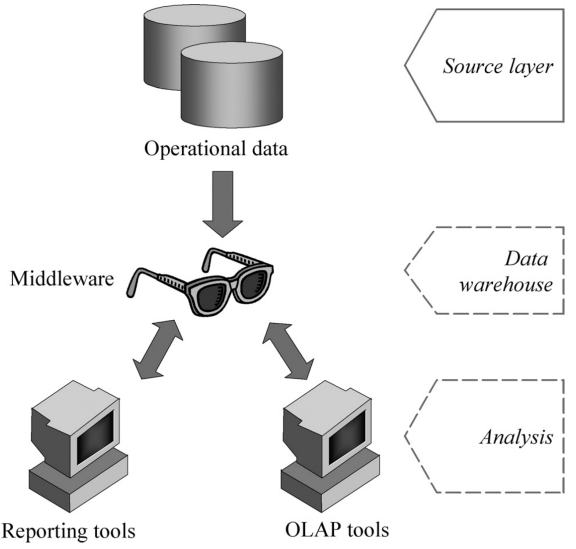


Figura 2.1: Almacén de datos. Arquitectura de una capa.

Arquitectura de dos capas

Fue diseñada con el objetivo de solucionar el problema de la separación que presentaba la arquitectura de una capa. Este esquema consigue **subrayar la separación entre los datos disponibles y el almacén de datos** a través de los siguientes componentes (ver figura 2.2):

- **Capa de origen (fuente):** Se corresponde con los orígenes y fuentes de los datos heterogéneos que se pretenden incorporar al almacén de datos.
- **Puesta a punto:** Proceso por el cual se utilizan herramientas de Extracción, Transformación y Carga (ETL) para extraer, limpiar, filtrar, validar y cargar datos en el almacén de datos.
- **Capa de almacén de datos:** Almacenamiento centralizado de la información en el almacén de datos, el cual puede ser utilizado para crear *data marts* o repositorios de metadatos.
- **Análisis:**. Conjunto de procesos a partir de los cuales los datos son eficientemente y flexiblemente analizados, generando informes y simulando escenarios hipotéticos para dar soporte a la toma de decisiones.



Un **Data mart** es un subconjunto o agregación de los datos almacenados en un almacén de datos primario que incluye información relevante sobre un área específica del negocio (ver [PVMCV06], [GR09]).

Arquitectura de tres capas

Este tercer tipo de arquitectura incluye una capa llamada de **datos reconciliados o almacén de datos operativos.** Con esta capa, los datos operativos obtenidos tras la limpieza y depuración son integrados y validados, proporcionando un modelo de datos de referencia para toda la organización. De este modo, **el almacén de datos no se nutre de los datos de origen directamente, sino de los datos reconciliados generados**, los cuales también son utilizados para realizar de forma más eficiente tareas operativas, como la realización de informes o la alimentación de datos a procesos operativos. La figura 2.3 muestra de forma gráfica este tipo de arquitectura. Esta capa de datos reconciliados también puede implementarse de forma virtual en una arquitectura de dos capas, ya que se define como una vista integrada y coherente de los datos de origen.

2.3.2. Arquitecturas orientadas a la empresa

Esta clasificación distingue **cinco tipos de arquitecturas** que combinan las capas mencionadas en la primera clasificación para diseñar almacenes de datos (ver [JVVL03], [GR09]).

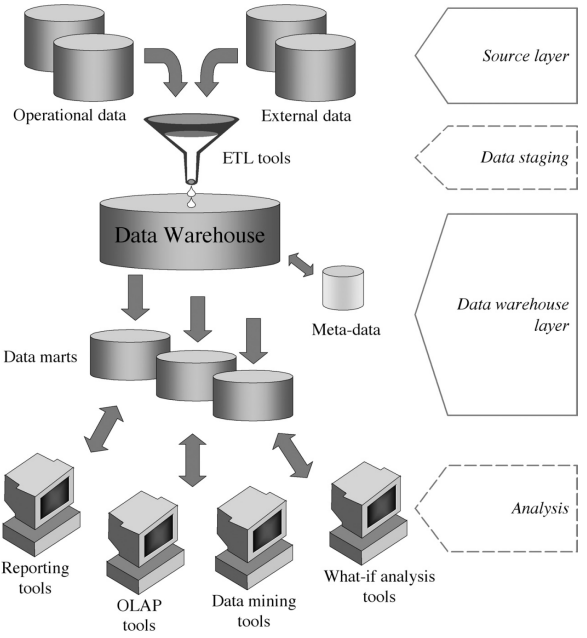


Figura 2.2: Almacén de datos. Arquitectura de dos capas.

Arquitectura de *data marts* independientes

Arquitectura preliminar en la que **los distintos *data marts* son diseñados de forma independiente y construidos de forma no integrada.** Suele utilizarse en los inicios de implementación de proyectos de almacenes de datos y reemplazada a medida que el proyecto va creciendo.

Arquitectura en bus

Similar a la anterior, **asegura la integración lógica de los *data marts* creados**, ofreciendo una visión amplia de los datos de la empresa y permitiendo realizar análisis rigurosos de los procesos que en ella se llevan a cabo.

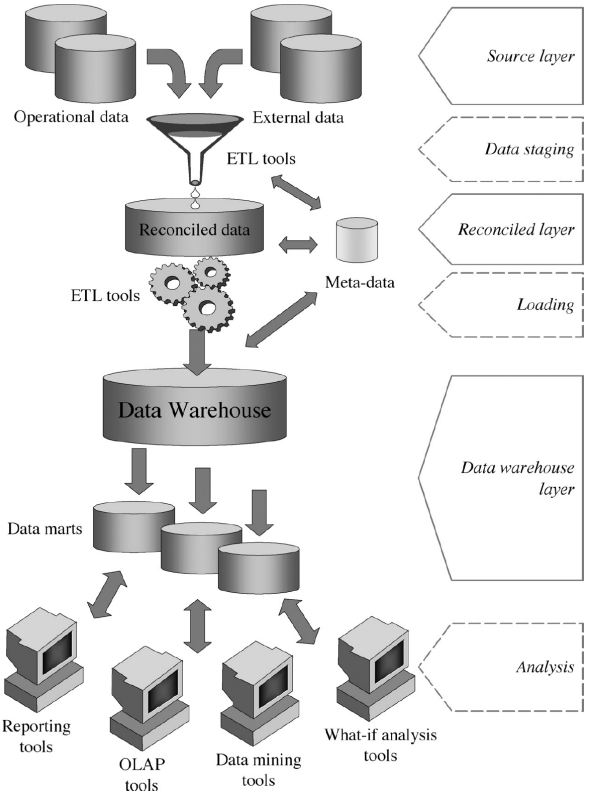


Figura 2.3: Almacén de datos. Arquitectura de tres capas.

Arquitectura hub-and-spoke (centro y radio)

Esta arquitectura es **muy utilizada en almacenes de datos de tamaños medio y grande.** Su diseño pone especial énfasis en garantizar la escalabilidad del sistema y permitir añadir extensiones al mismo. Para ello, **los datos se almacenan de forma atómica y normalizada en una capa de datos reconciliados que alimenta a los *data marts*** construidos que contienen, a su vez, los datos agregados de forma multidimensional. Los usuarios acceden a los *data marts*, si bien es cierto que también pueden hacer consultas directamente sobre los datos reconciliados.

Arquitectura centralizada

Se trata de un caso particular de la arquitectura hub-and-spoke. En ella, **la capa de datos reconciliados y los *data marts* se almacenan en un único repositorio físico.**

Arquitectura federada

Se trata de un tipo de arquitectura **muy utilizada en entornos dinámicos, cuando se pretende integrar almacenes de datos o *data marts* existentes con otros para ofrecer un entorno único e integrado de soporte a la toma de decisiones.** De esta forma, cada almacén de datos y cada *data mart* es integrado virtual o físicamente con lo demás. Para ello, se utilizan una serie de técnicas y herramientas avanzadas como son las ontologías, consultas distribuidas e interoperabilidad de metadatos, entre otras.

2.4. Almacenes de datos: Diseño e implementación

En esta sección se profundizará en cada una de las **etapas necesarias para diseñar e implementar un almacén de datos.** Para ello, en primer lugar se describirá el proceso de Extracción, Transformación y Carga (ETL), fundamental para construir y alimentar un almacén de datos. Después, se desarrollarán dos de los diseños más extendidos de almacén de datos: **el diseño en estrella y el diseño en copo de nieve.**

2.4.1. El proceso de Extracción, Transformación y Carga (ETL)

El proceso ETL es el encargado de **extraer, limpiar e integrar los datos provenientes de las fuentes de datos para alimentar el almacén de datos.** Este proceso también es el encargado de de alimentar la capa de datos reconciliados en la arquitectura de tres capas. El proceso ETL **tiene lugar cuando se puebla el almacén de datos y se lleva a cabo cada vez que el almacén de datos se actualiza.** A continuación, se describen detalladamente cada una de las fases de las que consta este proceso (ver [KR02],[MTK06] ,[GR09]).

Extracción

Etapa que consiste en la **lectura de los datos de las distintas fuentes de las que provienen.** Cuando un almacén de datos se rellena por primera vez, se suele utilizar la técnica de **extracción estática**, la cual consiste en extraer una instantánea de los datos operacionales. A partir de entonces, se utiliza la **extracción incremental** para actualizar periódicamente los datos del almacén de datos, recogiendo los cambios aplicados desde la última extracción. Para ello, se utiliza el registro mantenido por el SGBD que, por ejemplo, asocia una **marca de tiempo (*timestamp*)** a los datos operacionales para registrar cuando fueron modificados y agilizar el proceso de extracción.

En la actualidad, existe una gran cantidad de conjuntos de datos o *data sets* públicos, conocidos bajo el nombre de *Open Data*, que abarcan una gran cantidad de dominios y con los que es posible trabajar para construir soluciones big data.



Open Data: Se trata de datos que han sido generados por una fuente en particular, que abarcan un dominio temático o disciplinar y tienen atributos, dentro de los cuales está la frecuencia de actualización. Además, cuentan con una licencia específica que indica las condiciones de reutilización de los mismos.

La fuente de los datos es en muchos de los casos el estado nacional, provincial, municipal u organizaciones comerciales. En otras ocasiones, **la fuente de los datos es fruto del estudio o medición por parte de particulares.** Los **atributos** de los conjuntos de datos deben especificar cómo fueron obtenidos, incluyendo fechas de obtención, actualización y validez, así como el público involucrado, la metodología de recogida o muestreo, etc.

Algunas de las fuentes más utilizadas en la actualidad para la obtención de datos abiertos provienen de **los centros nacionales e internacionales de estadísticas**, como son el Instituto Nacional de Estadística de España (INE)[1], eurostat[2], la oficina europea de estadísticas, la Organización Mundial de la Salud (OMS)[3]... Por otra parte, existen **repositorios públicos de datos abiertos y a disposición de los usuarios como UCI**[4] **o Kaggle**[5], que es una comunidad de científicos de datos donde empresas y organizaciones suben sus datos y plantean problemas que son resueltos por los miembros de la comunidad. La figura 2.4 muestra las fuentes de datos que se acaban de mencionar.



Figura 2.4: Fuentes para la obtención de datos abiertos.

[1] https://www.ine.es
[2] https://ec.europa.eu/eurostat
[3] https://www.who.int/es/data/gho/publications/world-health-statistics
[4] https://archive.ics.uci.edu/ml/index.php
[5] https://www.kaggle.com

Transformación

La etapa de transformación es la fase clave para **transformar los datos operativos en datos con un formato específico para alimentar un almacén de datos.** En esta etapa, **los datos se limpian y se transforman, añadiéndoles contexto y significado.** En caso de implementar un almacén de datos siguiendo una arquitectura de tres capas, **el proceso de transformación es el encargado de obtener la capa de datos reconciliados.** Si bien es cierto que algunos autores separan la limpieza y la transformación de los datos en dos etapas distintas, en este capítulo se considerarán ambas dentro de la fase de transformación.



La etapa de **transformación** engloba todos los procesos de limpieza y manipulación de los datos, con el objetivo de transformar los datos operativos propios de sistemas relacionales (OLTP) en datos preparados para ser incluidos dentro del almacén de datos (OLAP).

La limpieza de los datos o *data cleaning* engloba todos aquellos procedimientos necesarios para detectar y resolver situaciones problemáticas con los datos de partida que pudieran suponer problemas potenciales a la hora de analizarlos. Así pues, los datos de partida pueden ser **incompletos**, es decir, pueden contener atributos sin valor o valores agregados, **incorrectos** que incluyan errores o valores sin ningún significado, lo cual es común cuando los datos se introducen manualmente en el sistema, o **inconsistentes** cuando los cambios no son propagados a todos los módulos del sistema, los rangos de un determinado atributo son cambiantes, existen datos duplicados...

Otra tarea muy importante que se debe abordar en la fase de limpieza es la gestión de los **datos perdidos**. Se trata de **datos que no están disponibles para algunas de las variables en alguna instancia**. Esto puede ser debido, por ejemplo, a que no se registraran las modificaciones sufridas por los datos, se produjera un mal funcionamiento del equipo, los datos nunca fueron rellenados o no existían en el momento en que fueron completados, se eliminaron por ser inconsistentes con la información registrada...

En general, se distinguen **dos tipos de situaciones cuando existen valores perdidos: datos perdidos completamente aleatorios y datos perdidos de forma no completamente aleatoria.** En el segundo caso, puede ser interesante intentar analizar la razón de la pérdida de los datos, la cual puede ser indicativa. En muchas ocasiones, los valores perdidos tienen relación con un subconjunto de variables predictoras y no se encuentran aleatoriamente distribuidos por todas ellas. Por todo ello, las aproximaciones más comunes a la hora de gestionar datos perdidos son:

- **Eliminación de instancias que contengan valores perdidos:** consiste en establecer un porcentaje umbral de tal forma que, si una instancia contiene un número de valores perdidos que supera este porcentaje, la instancia se descarta. Se trata de una técnica útil cuando el conjunto de datos es grande y el número de instancias con valores perdidos no es muy elevado. No obstante, esta estrategia se debe utilizar con precaución, ya que puede suponer una excesiva pérdida de información si el conjunto de datos no es muy grande, el número de instancias con valores perdidos es alto o el porcentaje umbral es muy pequeño.
- **Asignación de valores fijos:** consiste en asignar un valor fijo a todos los valores perdidos de todas las variables. Este valor puede ser el número 0 o incluso un valor desconocido *Unknown* o no numérico (NaN, Not a Number) en función del lenguaje de programación utilizado.
- **Asignación de valores de referencia:** asigna un valor de referencia a los valores perdidos para cada variable. Estos valores de referencia suelen ser medidas de centralización como la media o la mediana de los valores de cada variable.
- **Imputación de valores perdidos:** consiste en la aplicación de técnicas más sofisticadas, como pueden ser técnicas estadísticas o de aprendizaje automático para predecir o averiguar los valores que se han perdido.

Finalmente, la etapa de limpieza de datos también se encarga de la **detección de valores anómalos o *outliers***. Se trata de valores que se han introducido de forma errónea o bien a una deformación en la distribución de valores. El proceso de detección de anomalías consiste, fundamentalmente, en dos etapas: en primer lugar, **asumir que existe un modelo generador de datos**, como podría ser una distribución de probabilidad. En segundo lugar, considerar que **las anomalías representan un modelo generador distinto**, que no coincide con el original. Existen multitud de técnicas para detectar y descartar o imputar valores anómalos, como lo son técnicas estadísticas basadas en la desviación y el rango intercuartílico o técnicas de aprendizaje automático.

Después de la etapa de limpieza, comienza la etapa de transformación. En ella, los datos se preparan para su carga en el almacén de datos. Para ello, se transformarán los datos provenientes de sistemas transaccionales OLTP en datos preparados para su análisis OLAP. A continuación, se muestran algunos de los procesos de transformación más comunes:

- **Estandarización de códigos y formatos de representación:** incluye tareas como transformar la información EBCDIC a formato ASCII o Unicode, convertir números cardinales en ordinales o viceversa, homogeneización y tratamiento de fechas, expandir codificaciones añadiendo textos descriptivos, unificación de códigos (sexo, estado civil, etc) y estándares (medidas, monedas, etc).

- **Conversiones y combinaciones de campos:** realización de agregaciones y cálculos simples (importe neto, beneficio, realización de cuentas y promedios), cálculos derivados con valores numéricos y textuales...
- **Correcciones:** en caso de que no se pudiera hacer en el origen o en la etapa de limpieza, se deben corregir errores tipográficos, datos sin sentido ni significado, resolver conflictos de dominio de variables, aclaración de datos ambiguos y asignación de valores a datos nulos.
- **Integración de varias fuentes:** implica la resolución de claves y utilización de diccionarios de datos y repositorios de metadatos para el diseño del almacén de datos.
- **Eliminación de datos y/o registros duplicados:** frecuente cuando se trabaja con distintas fuentes de datos provenientes de diferentes dimensiones o departamentos de la organización. Para ello, es muy útil el uso de funciones de búsqueda y agrupamiento aproximado.
- **Escalado y centrado:** para reducir los efectos adversos de contar con datos dispersos, es muy útil utilizar técnicas para escalar y centrar los datos. Una posible transformación consiste en restar a cada valor de cada instancia el valor medio de la variable correspondiente y después dividir los valores por la desviación estándar. De esta forma, los valores se escalan y se centran entorno al cero. Esta transformación también puede hacerse entorno al valor de la media o la mediana. También es posible escalar los valores en el intervalo $[0-1]$ o aplicar el logaritmo, para reducir la dispersión de los datos y mejorar la visualización.
- **Discretización:** la aplicación de muchas técnicas de aprendizaje automático requiere que los valores de las variables sean discreto, en lugar de continuos. La discretización es el proceso por el cual se divide un intervalo de valores continuos en tantos fragmentos como etiquetas o valores discretos se quieran asignar, sustituyendo los valores originales por la etiqueta o valor discreto correspondiente.
- **Selección de características:** permite reducir el coste computacional de trabajar con todas las variables del conjunto de datos cuando, en muchas ocasiones, existen variables que no aportan información para el aprendizaje. Además de reducir el coste computacional, se reduce el número de dimensiones del conjunto de datos, mejorando la visualización y mejorando el rendimiento de los modelos no solo en términos de tiempo, sino también de rendimiento, puesto que existen métodos de aprendizaje cuyo rendimiento es peor cuando se ejecutan utilizando variables no significativas.

Carga

Se trata de la última fase de cara a incluir datos en el almacén de datos. **La carga inicial de los datos puede requerir bastante tiempo al cargar de forma progresiva todos los datos históricos**, por lo que es normal realizarla en horas de baja carga de los sistemas. Una vez que el almacén de datos ha sido inicialmente cargado, las sucesivas cargas de datos se pueden realizar de dos formas (ver [GR09]):

- **Refresco:** El almacén de datos se reescribe completamente, de forma que se reemplazan los datos antiguos. El refresco **es utilizado habitualmente junto con la extracción estática** y suele ser una estrategia muy utilizada para **la carga inicial** del almacén de datos, aunque puede también realizarse a posteriori.
- **Actualización: Se añaden al almacén de datos solamente aquellos datos nuevos que se pretenden incluir, sin eliminar ni modificar los datos ya existentes.** Esta técnica es **utilizada frecuentemente junto con la extracción incremental** para actualizar regularmente el almacén de datos. Se trata de una estrategia de carga **compleja de diseñar**, ya que requiere que sea eficiente computacionalmente. Para ello, se requieren mecanismos de detección del cambio como aquellos basados en la marca de tiempo (*timestamp*) o la utilización de tablas de log, entre otros.

Frameworks ETL

Recientemente, han aparecido múltiples *frameworks* que ofrecen herramientas tecnológicas para dar un soporte integrado al proceso ETL. A continuación, se describen algunos de los más utilizados:

- **Bubbles (`http://bubbles.databrewery.org`):** Se trata de un *framework* ETL escrito en Python, aunque es posible utilizarlo desde otros lenguajes. Ofrece un marco de trabajo sencillo, donde el proceso ETL se describe como una secuencia de nodos conectados que representan los datos y las distintas operaciones que se pueden realizar con ellos. De esta forma, es posible construir un grafo que implemente el proceso ETL completo para un conjunto de datos.
- **Apache Camel (`https://camel.apache.org`):** Este *framework* escrito en lenguaje Java y de acceso abierto se enfoca en facilitar la integración entre distintas fuentes de datos, haciendo el proceso más accesible a los desarrolladores, ofreciendo disitntas herramientas para dar soporte al proceso ETL.
- **Keetle (`https://community.hitachivantara.com/s/article/data-integration-kettle`):** Es el *framework* de Pentaho para dar soporte al proceso ETL. Pentaho es una *suite* o conjunto de programas para construir soluciones de inteligencia de negocio y Keetle es su entorno de trabajo ETL. Similar a Bubbles, ofrece un entorno de trabajo sencillo para implementar un proceso ETL a través de nodos y conexiones entre ellos. Además, Keetle es de acceso abierto.

2.4.2. Diseño en Estrella

A la hora de diseñar un almacén de datos, existen dos alternativas ampliamente utilizadas: el **diseño en estrella**, que promueve el diseño directo de estructuras lógicas sobre el modelo relacional, y **el diseño en copo de nieve**, como variante del diseño en estrella.

El proceso de desarrollo de un almacén de datos siguiendo el diseño en estrella puede estructurarse según las siguientes fases (ver [KR02],[PVMCV06]):

1. **Elegir un proceso de negocio a modelar:** cualquier proceso de negocio puede ser modelado como un almacén de datos. No obstante, serán de especial interés aquellos procesos **necesarios para la toma de decisiones y que involucran a diferentes unidades de la organización.**

2. **Escoger la granularidad del proceso de negocio:** Los datos almacenados en el almacén de datos **deben expresarse siempre al mismo nivel de detalle.** Por este motivo es necesario escoger un nivel de granularidad al comienzo del diseño del almacén de datos. En caso de que se pretenda almacenar información relevante con distintos niveles de granularidad, esta información deberá almacenarse de forma separada.

3. **Selección de medidas/hechos:** indican **qué se necesita medir o evaluar en el proceso de negocio** con el fin de dar respuesta a las necesidades de información y toma de decisiones por las cuales se pretende diseñar el almacén de datos. Por tanto, constituyen qué se quiere analizar. Las medidas/hechos son numéricas y es posible agregarlas a distintos niveles de detalle. Por ejemplo, en el ámbito logístico las medidas podrían ser las unidades aceptadas o devueltas, mientras que en el ámbito del control de calidad podrían ser las unidades producidas, defectuosas, costo de la producción.

4. **Elegir las dimensiones que se aplicarán a cada medida o hecho:** las dimensiones especifican **las distintas propiedades de las medidas o hechos que se pretenden almacenar e integrar dentro del almacén de datos.** Por ejemplo, si las medidas seleccionadas están relacionadas con las ventas de una determinada empresa, las dimensiones podrían ser: fecha de la venta, vendedor ,cliente, producto...

A la hora de diseñar el almacén de datos siguiendo el diseño en estrella, se crea una **tabla central, también llamada tabla de hechos, tabla factual o *fact table***. Esta tabla es la que posee los datos (medidas o hechos) sobre las diferentes combinaciones de las dimensiones. En algunas ocasiones, un diseño en estrella presenta más de una tabla de hechos. Los hechos introducidos pueden ser de distintos tipos: **completamente aditivos** (total de ventas de un departamento), **no aditivos** (márgen de beneficios expresado como porcentaje) o **semiaditivos** (como datos intermedios que pueden agregarse con datos de otras dimensiones).

Rodeando a la tabla de hechos, se encuentra **una tabla por cada una de las dimensiones definidas.** La clave primaria de la tabla de hechos se crea combinando las claves primarias de sus dimensiones relacionadas, de forma que está compuesta por las claves ajenas de las tablas de dimensiones que relaciona. De esta forma, la tabla de hechos presenta una relación del tipo "muchos a muchos" entre todas las tablas de dimensiones que relaciona, a la vez que una relación "muchos a uno" con cada tabla de dimensión por separado. Este esquema con forma de estrella da nombre a este tipo de diseño (ver [KR02],[MTK06]).

Sea un almacén de datos en el que se pretende almacenar **las ventas llevadas a cabo en una organización.** La tabla central de hechos representa los datos de cada venta, mientras que las dimensiones que rodean a la tabla central recogen datos sobre los productos vendidos, la fecha de la venta, el canal de distribución y el lugar de entrega. La figura 2.5 muestra un ejemplo de este esquema de almacén de datos.

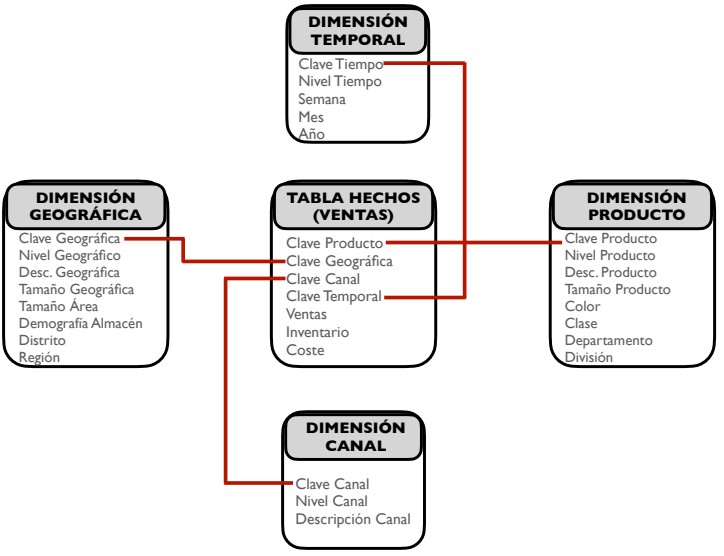


Figura 2.5: Ejemplo de almacén de datos diseñado en estrella.

Entre las **ventajas del diseño en estrella**, destaca ser un diseño **fácil de modificar que proporciona tiempos rápidos de respuesta**, simplificando la navegación de los medatadatos. Además, este diseño **permite simular vistas de los datos que obtendrán los usuarios finales y facilita la interacción con otras herramientas.**

Sin embargo, el diseño en estrella presenta algunos **inconvenientes** que surgen, por ejemplo, cuando se **combinan dimensiones con distintos niveles de detalle.** Además, cuando se pretende limitar los niveles de una dimensión se debe añadir un campo de nivel o utilizar un modelo de tipo constelación, donde se incluyen diferentes estrellas que almacenan tablas de hechos agregadas, lo cual **añade complejidad al esquema.**



El diseño en estrella permite crear un almacén de datos con una estructura centralizada. El diseño se compone de una tabla de hechos central, que recoge los valores de las medidas del proceso de negocio que se pretende modelar. Rodeando a la tabla de hechos, se incluyen tantas tablas como dimensiones se hayan especificado en el modelo, las cuales se relacionan entre sí a través de la tabla de hechos.

2.4.3. Diseño en Copo de Nieve

Otro de los diseños más extendidos a la hora de implementar un almacén de datos es el **diseño en copo de nieve.** De hecho, el diseño de copo de nieve es considerado una variante o derivado del diseño en estrella y, por tanto, puede construirse a partir de este siguiendo las mismas etapas que se describieron en la sección anterior.

Este diseño es muy utilizado cuando se dispone de **tablas de dimensión con una gran cantidad de atributos.** En esta circunstancia, el diseño de copo de nieve permite **normalizar las tablas de dimensión,** ofreciendo un mejor rendimiento cuando las consultas realizadas sobre el almacén de datos requieren del uso de operadores de agregación. De esta forma, la tabla de hechos deja de ser la única tabla que presenta relaciones con las demás, apareciendo nuevas relaciones que se dan entre las tablas normalizadas que componen las tablas de dimensiones. Este esquema, que incluye ramificaciones en cada dimensión que se corresponden con las tablas necesarias para su normalización, guarda un gran parecido con la estructura de un copo de nieve, lo cual da nombre a este diseño (ver [KR02], [MTK06]).

La diferencia entre los diseños en estrella y copo de nieve radica fundamentalmente en la modelización de las tablas de dimensiones. Para obtener un diseño en copo de nieve, basta con partir de un diseño en estrella en el que, tras un proceso de normalización, se obtienen varias tablas de dimensión a partir de una tabla desnormalizada. Dentro del diseño de copo de nieve, es posible distinguir entre **copo de nieve parcial,** en el que solo algunas de las dimensiones son normalizadas y **copo de nieve completo,** en el que todas las dimensiones del esquema son normalizadas (ver [JVVL03],[PVMCV06]).

Siguiendo con el ejemplo anterior, en el que se mostraba un almacén de datos para representar las ventas realizadas por una organización, la figura 2.6 muestra un diseño para un almacén de datos siguiendo un esquema de copo de nieve parcial, en el que las dimensiones geográfica y producto han sido normalizadas.

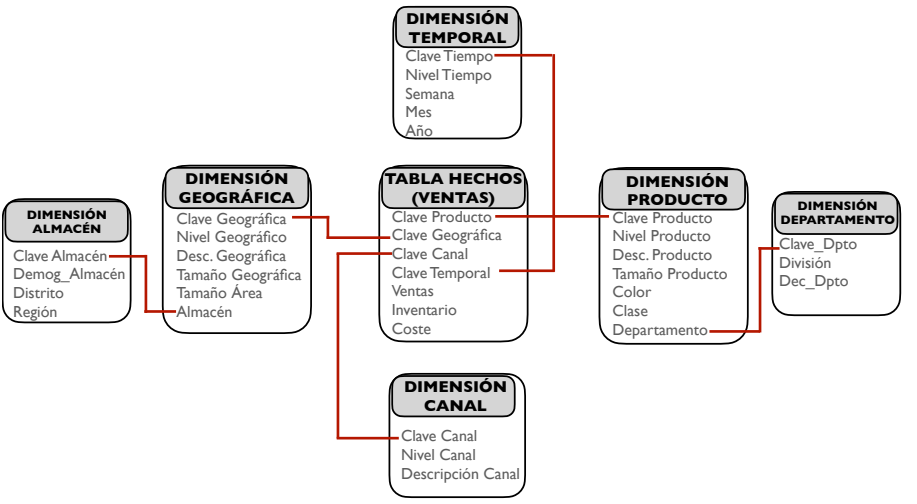


Figura 2.6: Ejemplo de almacén de datos diseñado en copo de nieve.

La normalización de las tablas de dimensiones proporciona al diseño de copo de nieve la **mejora de la eficiencia en la realización de consultas complejas que requieren el uso de operadores de agregación.** Sin embargo, este diseño es conceptualmente más difícil de implementar, ya que **incluye un mayor número de tablas** y requiere de realizar muchos *joins* entre ellas, lo que **incrementa los tiempos de consulta.**

En ocasiones, se requiere diseñar un almacén de datos que contenga más de una tabla de hechos. Para ello, **el esquema o diseño en constelación** permite crear un almacén de datos de forma similar al diseño en estrella, con la diferencia de que este modelo incluye más de una tabla de hechos. Además, cada tabla de hechos puede vincularse con todas o solo algunas tablas de dimensiones. Este esquema contribuye a la reutilización de las tablas de dimensiones, las cuales se pueden relacionar con más de una tabla de hechos.



El diseño en copo de nieve permite crear un almacén de datos a partir de un diseño en estrella. Para ello, las tablas de dimensiones se normalizan, generando más de una tabla para cada una de las dimensiones, mejorando la eficiencia de consultas complejas que requieren el uso de operadores avanzados.

Bibliografía

[AN95] Agnar Aamodt and Mads Nygård. Different roles and mutual dependencies of data, information, and knowledge — an ai perspective on their integration. *Data & Knowledge Engineering*, 16(3):191–222, 1995.

[BELP13] Ulrik Brandes, Markus Eiglsperger, Jürgen Lerner, and Christian Pich. Graph markup language (graphml). In Roberto Tamassia, editor, *Handbook on Graph Drawing and Visualization*, pages 517–541. Chapman and Hall/CRC, 2013.

[BNCD16] Rajkumar Buyya, Rodrigo N. Calheiros, and Amir Vahid Dastjerdi. *Big Data : Principles and Paradigms.* Elsevier Science & Technology, 50 Hampshire Street, 5th Floor, Cambridge. USA, 2016.

[dMP99] Adoración de Miguel and Mario Piattini. *Fundamentos y Modelos de Bases de Datos*. Alfaomega, 1999.

[FPSS96] Usama Fayyad, Gregory Piatetsky-Shapiro, and Padhraic Smyth. From data mining to knowledge discovery in databases. *AI magazine*, 17(3):37, 1996.

[Fri19] J. Friesen. *Java XML and JSON: Document Processing for Java SE*. Apress, 2019.

[GR09] Matteo Golfarelli and Stefano Rizzi. *Data Warehouse Design: Modern Principles and Methodologies*. Mcgraw-hill, 2009.

[HT14] Francisco Herrera Triguero. *Inteligencia Artificial, Inteligencia Computacional y Big Data*. Servicio de publicaciones. Universidad de Jaén, Jaén, 2014.

[IGG03] C. Imhoff, N. Galemmo, and J.G. Geiger. *Mastering Data Warehouse Design: Relational and Dimensional Techniques*. Wiley, 2003.

[Inm02] William H. Inmon. *Building the Data Warehouse,3rd Edition*. John Wiley & Sons, Inc., USA, 3rd edition, 2002.

[JVVL03] Matthias Jarke, Yannis Vassiliou, Panos Vassiliadis, and Maurizio Lenzerini. *Fundamentals of Data Warehouses*. Springer-Verlag, Berlin, Heidelberg, 1st edition, 2003.

[KR02] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit: The Complete Guide to Dimensional Modeling*. Wiley, 2 edition, April 2002.

[MSC13] Cukier Mayer Schönberger, Viktor and Kenneth Cukier. *Big data. La revolución de los datos masivos*. Turner Publicaciones, Madrid, 2013.

[MTK06] J. Mundy, W. Thornthwaite, and R. Kimball. *The Microsoft Data Warehouse Toolkit: With SQL Server 2005 and the Microsoft Business Intelligence Toolset*. Wiley, 2006.

[PVMCV06] Mario Piattini Velthuis, Esperanza Marcos, Coral Calero, and Belén Vela. *Tecnología y diseño de bases de datos*. Ra-Ma,, 2006.

[RB02] Pablo Rodríguez Bocca. Lenguajes canónicos para la descripción de grafos: Estudio y transformación entre esquemas de distintos modelos de datos. Interoperabilidad, Monografía, 2002.

[RWE15] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph Databases*. O'Reilly, 2 edition, 2015.

[SKS06] Abraham Silberschatz, Henry F. Korth, and Sudarshan S. *Fundamentos de bases de datos*. McGraw-Hill, Edificio Valrealty. 1 Planta. Basauri 17. 28023 Aravaca (Madrid), 2006.

[TR03] Erik T. Ray. *Learning XML*. O'Reilly, 2003.